

Transportation Problems

(and linear programming)

David Pisinger

`pisinger@man.dtu.dk`

DTU Management

Technical University of Denmark

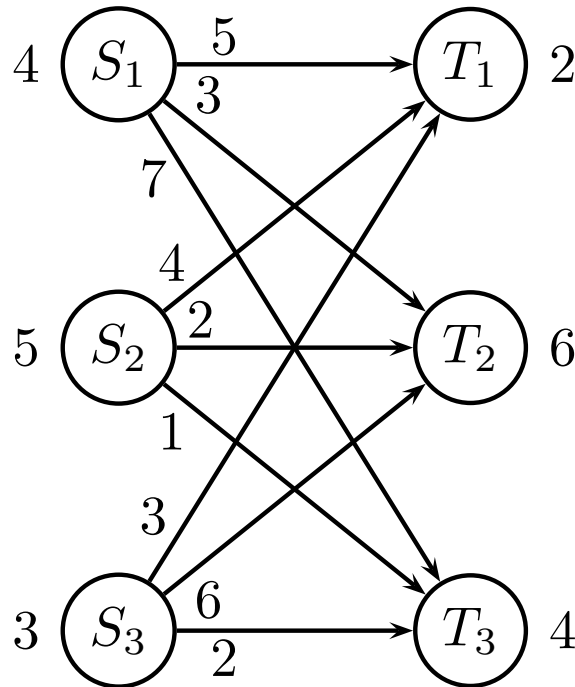
© Copyright David Pisinger. May not be distributed

Transportation Problem

- Bi-partite graph ($m + n$ nodes)
- Sources $S_1, \dots, S_m$, terminals $T_1, \dots, T_n$
- Supply s_i in S_i , demand d_j in T_j
- One commodity, various demands
- Edges have cost $c_{ij} \in \mathbb{R}$, infinite capacity
- Specialized network simplex
- **Next week:** Fixed charge
- Dantzig-Wolfe decomposition
- Column generation

Transportation Problem

Sources S_1, S_2, S_3 , terminals T_1, T_2, T_3



transportation costs:

c_{ij}	T_1	T_2	T_3
S_1	5	3	7
S_2	4	2	1
S_3	3	6	2

Transportation Problem, LP-model

Variable x_{ij} denotes flow from S_i to T_j

$$\min 5x_{11} + 3x_{12} + 7x_{13} + 4x_{21} + 2x_{22} + 1x_{23} + 3x_{31} + 6x_{32} + 2x_{33}$$

$$\text{s.t. } -x_{11} \quad -x_{12} \quad -x_{13} \quad \quad \quad = -4 \quad (u_1)$$

$$\quad \quad -x_{21} \quad -x_{22} \quad -x_{23} \quad \quad \quad = -5 \quad (u_2)$$

$$\quad \quad \quad -x_{31} \quad -x_{32} \quad -x_{33} = -3 \quad (u_3)$$

$$x_{11} \quad \quad \quad +x_{21} \quad \quad \quad +x_{31} \quad \quad = 2 \quad (v_1)$$

$$\quad x_{12} \quad \quad \quad +x_{22} \quad \quad \quad +x_{32} \quad \quad = 6 \quad (v_2)$$

$$\quad \quad x_{13} \quad \quad \quad +x_{23} \quad \quad \quad +x_{33} = 4 \quad (v_3)$$

$$x_{11}, x_{12}, x_{13}, x_{21}, x_{22}, x_{23}, x_{31}, x_{32}, x_{33} \geq 0$$

Solution of Trans. Problem

Solution method

- we can use LP to solve the problem
- network simplex (for transportation model) more interesting

Integer solutions

- constraint matrix is TU (Totally Unimodular)
- if demands are integers the LP-solution will always be integral

Solution of Transportation model

Negative demand for sources, positive for terminals. Model:

$$\begin{aligned} \min \quad & \sum_i \sum_j c_{ij} x_{ij} \\ \text{s.t.} \quad & - \sum_j x_{ij} = -s_i \quad \forall i = 1, \dots, m && u_i \\ & \sum_i x_{ij} = d_j \quad \forall j = 1, \dots, n && v_j \\ & x_{ij} \geq 0 \quad \forall i, j \end{aligned}$$

Assume $\sum_i s_i = \sum_j d_j$

Last terminal takes the rest.

One of the constraints is redundant

so we have only $m + n - 1$ basis variables

Dual

$$\begin{aligned} \max \quad & - \sum_i s_i u_i + \sum_j d_j v_j \\ \text{s.t.} \quad & -u_i + v_j \leq c_{ij} \quad \forall (i, j) \in E \\ & u_i, v_j \in \mathbb{R} \end{aligned}$$

Dual variables are not well-defined (several solutions)
Reduced cost for x_{ij} is found in row (0) in simplex

$$c_{ij} + u_i - v_j$$

Complementary slack

$$\begin{aligned}
 \min \quad & \sum_i \sum_j c_{ij} x_{ij} \\
 \text{s.t.} \quad & -\sum_j x_{ij} = -s_i \quad \forall i \quad \textcolor{red}{u_i} \\
 & \sum_i x_{ij} = d_j \quad \forall j \quad \textcolor{red}{v_j} \\
 & x_{ij} \geq 0 \quad \forall ij
 \end{aligned}$$

$$\begin{aligned}
 \max \quad & \sum_i -s_i u_i + \sum_j d_j v_j \\
 \text{s.t.} \quad & -u_i + v_j \leq c_{ij} \quad \forall ij \quad \textcolor{red}{x_{ij}} \\
 & u_i, v_j \text{ real}
 \end{aligned}$$

Complementary slack: $x_{ij} = 0 \quad \vee \quad -u_i + v_j = c_{ij}$

We demand: $x_{ij} > 0 \quad \Rightarrow \quad c_{ij} + u_i - v_j = 0$

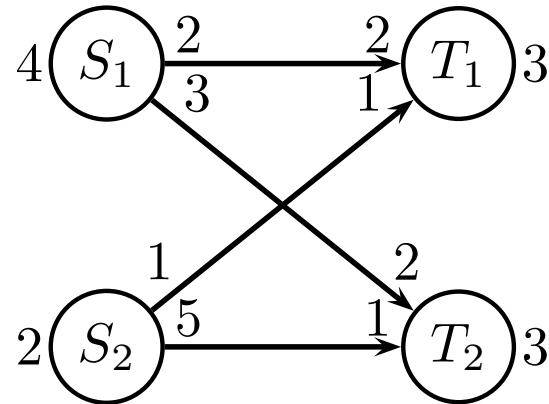
Solution method: Maintain primal feasible solution x_{ij} .

Improve solutions by considering **cycles**

Guess dual solution u_i, v_j satisfying complementary slack

When dual solution is legal, optimality is reached

Improving cycles



- Demands are still satisfied
- When traversing edge backward, we “cancel” flow
- We can only cancel as much as we currently send

Simplex

Z	x	x_s	
1	c	0	0
	A	I	b
1	$c - y^* A$	y^*	$y^* b$
	$S^* A$	S^*	$S^* b$

first tableau

last tableau

● $c - y^* A \geq 0$

● $y^* \geq 0$

● $S^* b = x_B^* \geq 0$

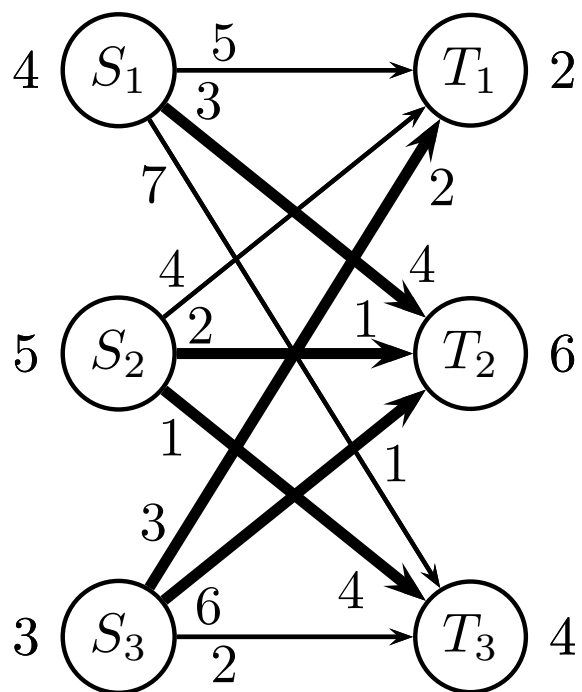
● canonic form: $c - y^* A = 0$ for basis variables

Network Simplex, Greedy heuristics

Heuristic: not optimal

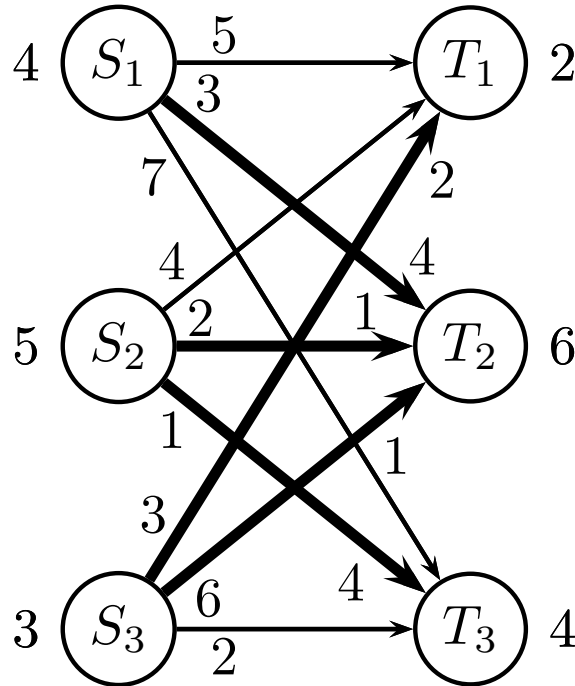
Greedy: chooses the most promising in current step

Consider sources S_1, S_2, S_3 one by one
transport items as cheap as possible to terminals



cost: $4 \cdot 3 + 1 \cdot 2 + 4 \cdot 1 + 2 \cdot 3 + 1 \cdot 6 = 30$

Network Simplex, Greedy heuristic



Thick edges if flow > 0 on edge

5 thick edges ($m + n - 1$) form **basis**

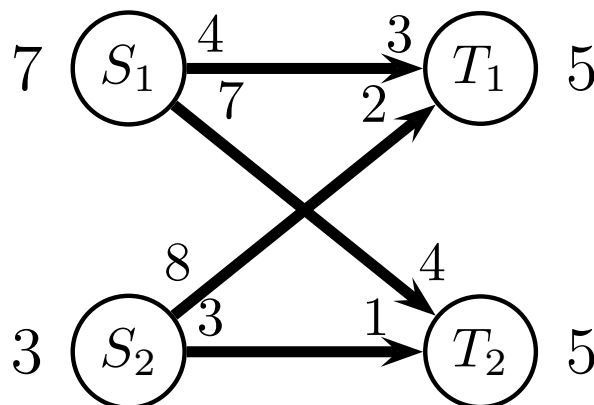
Basis is a **spanning tree** by construction

Algorithm will ensure tree in every iteration

If less than $m + n - 1$ edges have flow > 0 then choose extra edges with flow 0

Network Simplex, optimal solution is tree

Assume optimal solution is not a tree



E.g. cycle $C = \{S_1, T_1, S_2, T_2\}$

- $\text{cost}(C) > 0$: send some units backward
- $\text{cost}(C) < 0$: send some units forward
- $\text{cost}(C) = 0$: send some units forward or backward

So, an optimal solution exists without cycles

Network Simplex, iterations

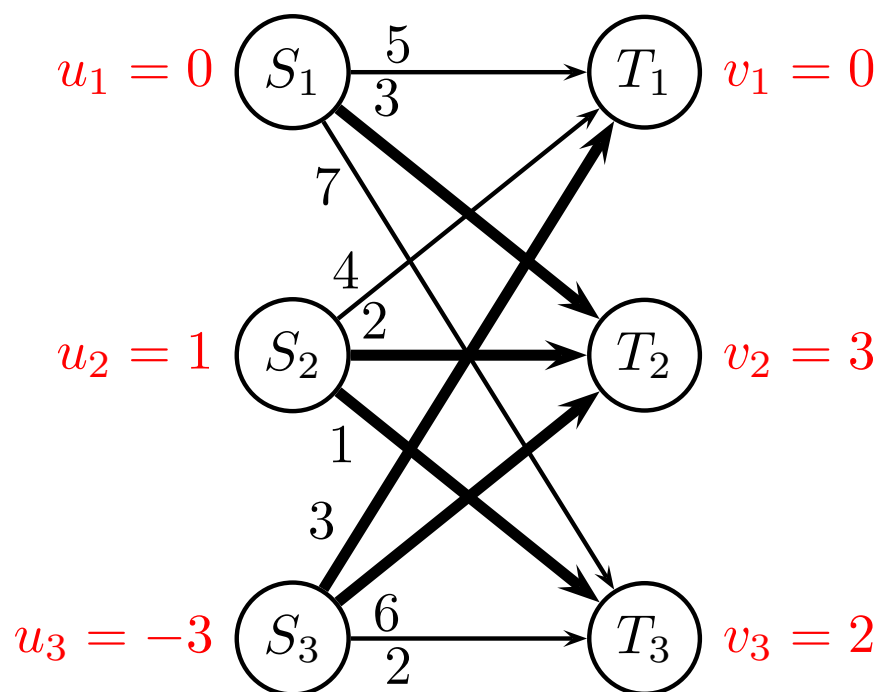
Choose node S_1 as root, calculate distance to nodes

Only thick edges may be used

→ cost positive

← cost negative

u_i = distance to S_i -nodes, v_j = distance to T_j -nodes



Network Simplex, iterations

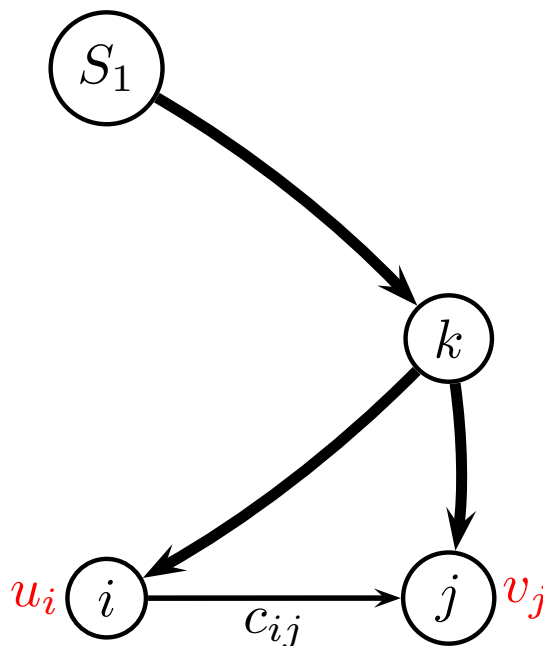
Find cycle in graph, saving transportation costs

- (S_1, T_1) impose cycle $(S_1, T_1, S_3, T_2, S_1)$, cost
 $5 - 3 + 6 - 3 = 5$
reduced cost $\bar{c}_{11} = c_{11} + u_1 - v_1 = 5 + 0 - 0 = 5$
- (S_1, T_3) impose cycle $(S_1, T_3, S_2, T_2, S_1)$, cost
 $7 - 1 + 2 - 3 = 5$
reduced cost $\bar{c}_{13} = c_{13} + u_1 - v_3 = 7 + 0 - 2 = 5$
- (S_2, T_1) impose cycle $(S_2, T_1, S_3, T_2, S_2)$, cost
 $4 - 3 + 6 - 2 = 5$
reduced cost $\bar{c}_{21} = c_{21} + u_2 - v_1 = 4 + 1 - 0 = 5$
- (S_3, T_3) impose cycle $(S_3, T_3, S_2, T_2, S_3)$, cost
 $2 - 1 + 2 - 6 = -3$
reduced cost $\bar{c}_{33} = c_{33} + u_3 - v_3 = 2 - 3 - 2 = -3$

Choose cycle with most negative reduced cost: edge (S_3, T_3)

Network Simplex, cost of cycle

Cost of cycle C imposed by edge (i, j) is equal to reduced cost $\bar{c}_{ij}$



$$\begin{aligned}\bar{c}_{ij} &= c_{ij} + u_i - v_j \\ &= c_{ij} + \text{dist}(S_1, k) + \text{dist}(k, i) - \text{dist}(S_1, k) - \text{dist}(k, j) \\ &= c_{ij} + \text{dist}(j, k) + \text{dist}(k, i) \\ &= \text{cost}(C)\end{aligned}$$

Network Simplex, iterations

How much can flow along cycle?

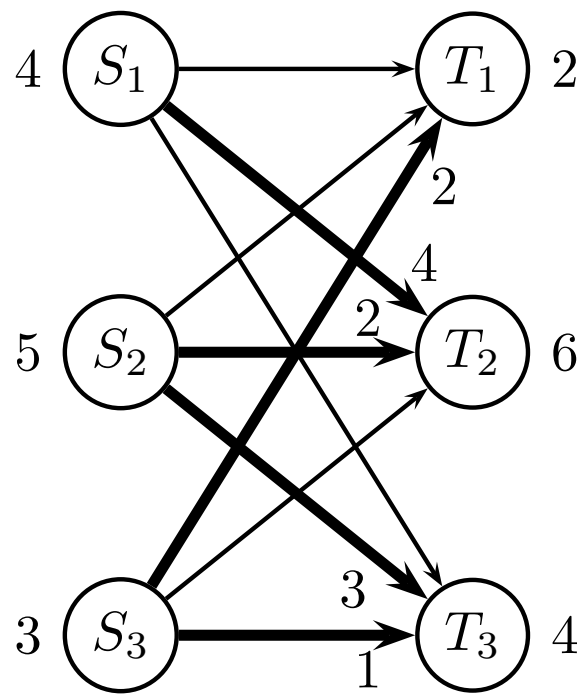
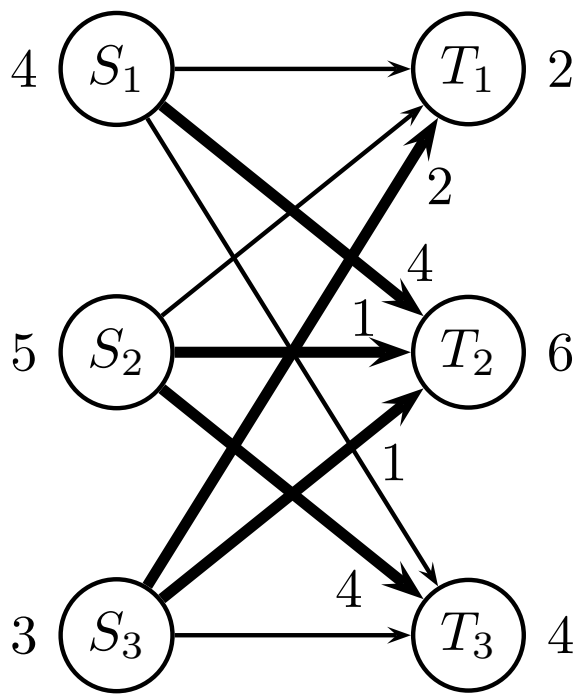
Forward edges: ∞ capacity

Backward edges: as much as current flow

Cycle $(S_3, T_3, S_2, T_2, S_3)$ has backward edges:

(T_3, S_2) flow 4
 (T_2, S_3) flow 1

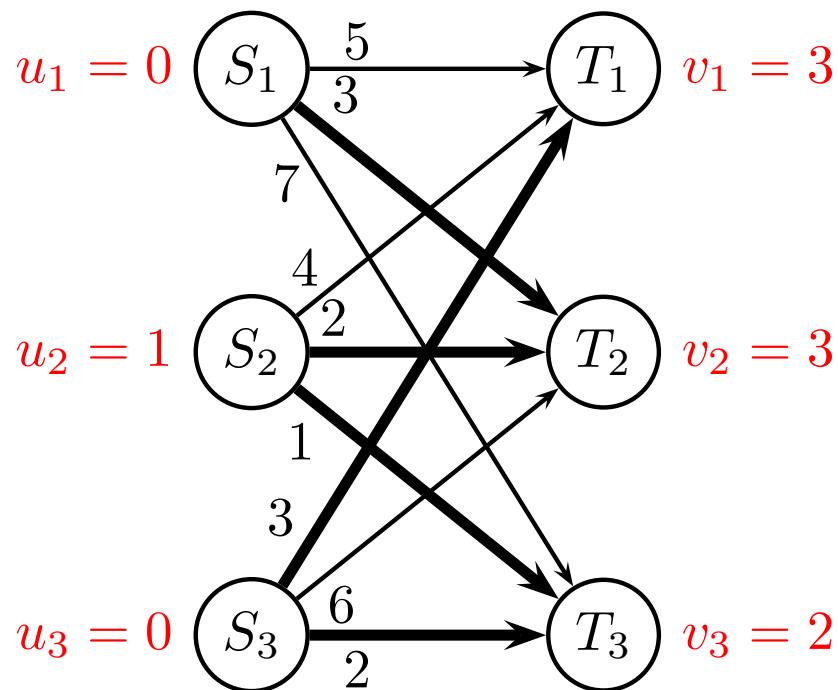
} send $\theta = 1$ unit



Cost: $4 \cdot 3 + 2 \cdot 2 + 3 \cdot 1 + 2 \cdot 3 + 1 \cdot 2 = 27$.

Network Simplex, iterations

Distances from S_1 to all other nodes in the tree:



Network Simplex, stop criteria

Find cycle in graph, saving transportation costs

- Edge (S_1, T_1) has $\bar{c}_{11} = c_{11} + u_1 - v_1 = 5 + 0 - 3 = 2$
- Edge (S_1, T_3) has $\bar{c}_{13} = c_{13} + u_1 - v_3 = 7 + 0 - 2 = 5$
- Edge (S_2, T_1) has $\bar{c}_{21} = c_{21} + u_2 - v_1 = 4 + 1 - 3 = 2$
- Edge (S_3, T_2) has $\bar{c}_{32} = c_{32} + u_3 - v_2 = 6 + 0 - 3 = 3$

Since no negative reduced cost, simplex terminates.

Reduced costs are:

- $\bar{c}_{ij} = 0$ for thick edges.
- $\bar{c}_{ij} \geq 0$ for thin edges.

This shows u, v is an optimal dual solution

Network Simplex, pseudocode

find greedy solution

let basis (fat edges) be those edges where flow > 0

if basis has less than $n + m - 1$ edges, add additional (thin) edges

repeat

- calculate dual costs u_i, v_j as distance (using fat edges) from node S_1
- for every thin edge calculate reduced cost as $\bar{c}_{ij} = c_{ij} + u_i - v_j$
- choose the thin edge (i', j') with most negative reduced cost
- define cycle C as (i', j') and fat edges
- the cycle C should move in the direction of (i', j')
- calculate the “bottleneck” θ as minimum of flow on backward edges in C
- send θ units along C
- edge (i', j') becomes fat, and the bottleneck edge becomes thin

until all reduced costs $\bar{c}_{ij} \geq 0$

Tips-and-tricks

Network simplex

- check that problem is balanced, otherwise add surplus node to make problem balanced
- if some edges are missing, you can add them with big cost M , this will ensure that you can find a greedy solution
- if $\theta = 0$ then just send zero items along the cycle, this will give a new basis

Network Simplex, running time

Let U be an upper bound on the objective, e.g.

$$U = \sum_i s_i \max_j c_{ij}$$

If demands and costs are integers, each iteration will decrease objective by at least 1

Each iteration takes:

- Find dual costs $O(E)$
- Find reduced costs for all thin edges $O(E)$
- Pivot operation $O(E)$

In total: $O(EU)$

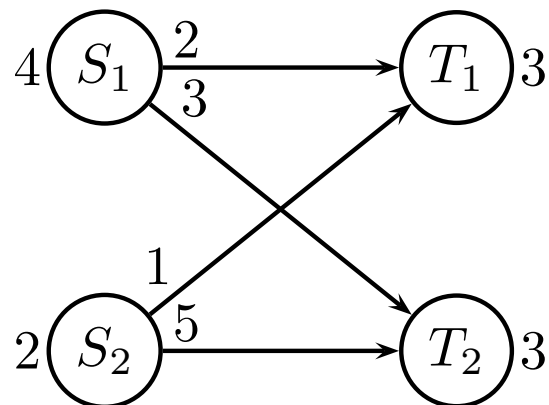
Relation to Simplex Algorithm

David Pisinger

`pisinger@man.dtu.dk`

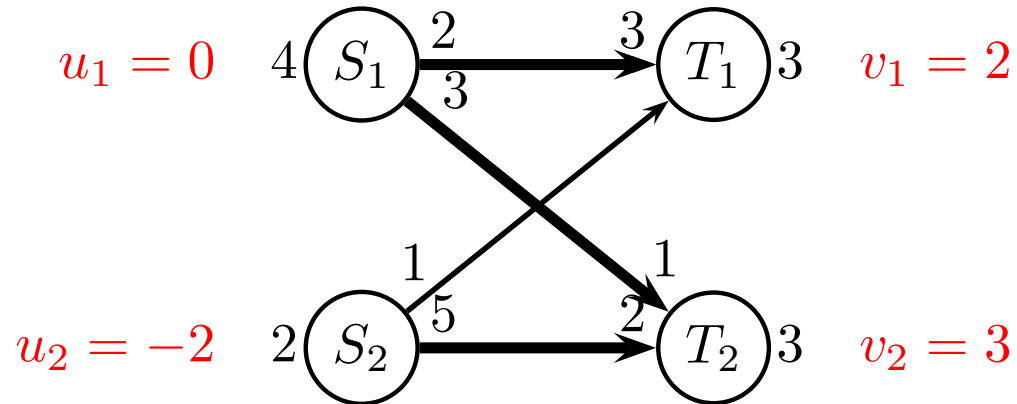
DTU Management
Technical University of Denmark

A Small Example



$$\begin{array}{llllllll}
 \min & 2 & x_{11} & + & 3 & x_{12} & + & 1 & x_{21} & + & 5 & x_{22} \\
 \text{s.t.} & -x_{11} & - & x_{12} & & & & & & & & = & -4 \\
 & & & & & & - & x_{21} & - & x_{22} & & = & -2 \\
 & x_{11} & & & & & + & x_{21} & & & & = & 3 \\
 & & x_{12} & & & & & & + & x_{22} & & = & 3
 \end{array}$$

A Small Example



Cost:

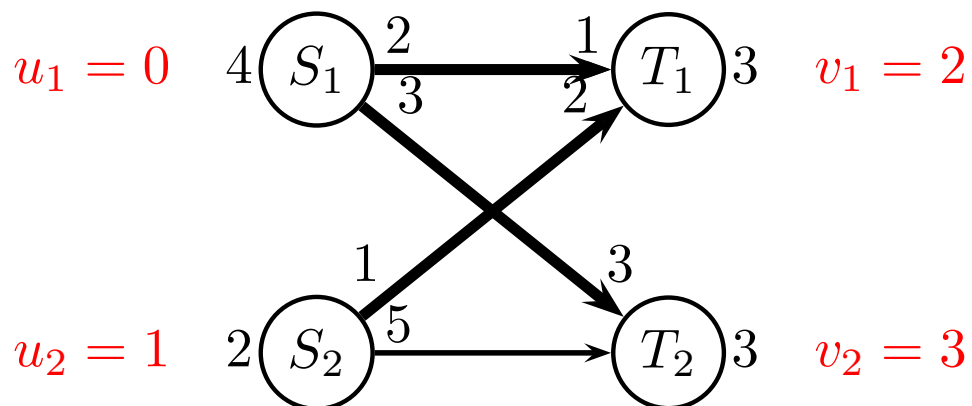
$$2 \cdot 3 + 3 \cdot 1 + 5 \cdot 2 = 19$$

Reduced cost:

$$\bar{c}_{21} = c_{21} + u_2 - v_1 = 1 - 2 - 2 = -3$$

Cycle S_2, T_1, S_1, T_2 , capacity $\min\{3, 2\} = 2$.

A Small Example



Cost:

$$2 \cdot 1 + 3 \cdot 3 + 1 \cdot 2 = 13$$

Reduced cost:

$$\bar{c}_{22} = c_{22} + u_2 - v_2 = 5 + 1 - 3 = 3$$

Stop

A Small Example

$$\begin{array}{rclcl}
 -Z + 2x_{11} + 3x_{12} + 1x_{21} + 5x_{22} & = & 0 & (Z) \\
 -x_{11} - x_{12} & = & -4 & (A) \\
 & -x_{21} - x_{22} & = & -2 & (B) \\
 & x_{11} + x_{21} & = & 3 & (C) \\
 & x_{12} + x_{22} & = & 3 & (D)
 \end{array}$$

Since $-(A)-(B)-(C) = (D)$, last row is redundant, can be deleted

$$\begin{array}{rclcl}
 -Z + 2x_{11} + 3x_{12} + 1x_{21} + 5x_{22} & = & 0 & (Z) \\
 -x_{11} - x_{12} & = & -4 & (A) \\
 & -x_{21} - x_{22} & = & -2 & (B) \\
 & x_{11} + x_{21} & = & 3 & (C)
 \end{array}$$

A Small Example

Should run two-phase algorithm, but in this case easy to guess solution with greedy algorithm.

Want basis x_{11}, x_{12}, x_{22}

$$\begin{array}{rcll}
 -Z + 2x_{11} + 3x_{12} + 1x_{21} + 5x_{22} & = & 0 & (Z) \\
 x_{11} + x_{12} & = & 4 & (A) \\
 x_{21} + x_{22} & = & 2 & (B) \\
 x_{11} + x_{21} & = & 3 & (C)
 \end{array}$$

Add $-(C)$ to (A)

$$\begin{array}{rcll}
 -Z + 2x_{11} + 3x_{12} + 1x_{21} + 5x_{22} & = & 0 & (Z) \\
 x_{12} - x_{21} & = & 1 & (A) \\
 x_{21} + x_{22} & = & 2 & (B) \\
 x_{11} + x_{21} & = & 3 & (C)
 \end{array}$$

A Small Example

Must ensure that basis variables have 0 in (Z)

$$-Z + 2x_{11} + 3x_{12} + 1x_{21} + 5x_{22} = 0 \quad (Z)$$

$$x_{12} - x_{21} = 1 \quad (A)$$

$$x_{21} + x_{22} = 2 \quad (B)$$

$$x_{11} + x_{21} = 3 \quad (C)$$

Add $-2(C)$ to (Z), add $-3(A)$ to (Z), and add $-5(B)$ to (Z)

$$-Z + \quad \quad \quad - 3x_{21} \quad \quad \quad = -19 \quad (Z)$$

$$x_{12} - x_{21} = 1 \quad (A)$$

$$x_{21} + x_{22} = 2 \quad (B)$$

$$x_{11} + x_{21} = 3 \quad (C)$$

Read basis solution: $x_{11} = 3, x_{12} = 1, x_{22} = 2$ and $z = 19$.

A Small Example

Reduced cost of x_{21} is -3, so x_{21} enters basis

$$\begin{array}{rclcl}
 -Z+ & & -3 & x_{21} & = -19 & (Z) \\
 & x_{12} & - & x_{21} & = 1 & (A) \\
 & & & x_{21} + x_{22} & = 2 & (B) \\
 & x_{11} & + & x_{21} & = 3 & (C)
 \end{array}$$

Coefficient test shows that (B) is most binding

Add (B) to (A), add -(B) to (C), and add 3(B) to (Z)

$$\begin{array}{rclcl}
 -Z+ & & +3 & x_{22} & = -13 & (Z) \\
 & x_{12} & + & x_{22} & = 3 & (A) \\
 & & x_{21} + & x_{22} & = 2 & (B) \\
 & x_{11} & - & x_{22} & = 1 & (C)
 \end{array}$$

No negative reduced cost, so stop.

Read basis solution: $x_{11} = 1, x_{12} = 3, x_{21} = 2$ and $z = 13$.

Relation to Simplex Algorithm

Similarities

- Basis (thick edges)
- Reduced costs
- Entering variable
- Pivot operation

Advantage

- Easy to construct initial solution
- More intuitive representation
- More compact

Complementary slack theorem (proof)

Assume that x is feasible to primal, and y is feasible to dual

$$\begin{array}{ll} \max & cx \\ \text{s.t.} & Ax \leq b \\ & x \geq 0 \end{array} \qquad \begin{array}{ll} \min & yb \\ \text{s.t.} & yA \geq c \\ & y \geq 0 \end{array}$$

If complementary slack is satisfied

$$(yA - c)x = 0 \quad \text{and} \quad y(b - Ax) = 0$$

then the solutions x, y are optimal.

Proof write out the above constraints

$$cx = yAx \quad \text{and} \quad yAx = yb$$

hence, $cx = yb$, implying that x and y are optimal ■

Totally Unimodular

Definition

An $m \times n$ integral matrix A is called *totally unimodular* (TU) if the determinant of each square submatrix of A is equal to 0, 1 or -1.

Obviously $a_{ij} \in \{0, 1, -1\}$

Recognising whether A is TU demands an exponential number of steps

Totally Unimodular

If A is TU then A^{-1} is integral

From Cramer's rule $A_{ij}^{-1} = C_{ji} / \det(A)$ where C_{ji} is the adjoint matrix

$$C_{ji} = (-1)^{i+j} \det(A_{\text{row } i, \text{ column } j \text{ removed}})$$

If A is TU and b integral, then every basis solution $x_B = A_B^{-1}b$ is integral

Totally Unimodular, sufficient condition

A matrix $A = (a_{ij})$ is TU if

1. $a_{ij} \in \{+1, -1, 0\}$ for all i, j .
2. Each column contains at most two nonzero coefficients
3. There exists a partition (P_1, P_2) of the set M of rows such that each column j containing two nonzeros satisfies:
 - coefficients have same sign $\Leftrightarrow$ rows belong to different sets
 - coefficients have different sign $\Leftrightarrow$ rows belong to same set

Totally Unimodular, examples

$$\begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}$$

$$\begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix}$$

$$\begin{pmatrix} 0 & 1 & -1 & 0 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$\begin{pmatrix} 1 & -1 & -1 & 0 \\ -1 & 0 & 0 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

$$\begin{pmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & -1 & 1 & 0 \\ -1 & 0 & 1 & 0 & -1 \\ 0 & 0 & 0 & 0 & -1 \\ 0 & -1 & 0 & 0 & 0 \end{pmatrix}$$

Upper matrices are not TU, lower matrices are TU.

Big-O notation

for Network Problems

David Pisinger

`pisinger@man.dtu.dk`

DTU Management

Technical University of Denmark

Complexity of Network Algorithm

A good algorithm should

- be fast
- use little space

Larger instances use more time/space

- n is size of instance (how many nodes/edges)
- measure running time and space as function of n

Independent of computer

- we count number of additions/multiplications/moves
- we do not care about constants

Complexity of Algorithm

Algorithm 1

n	time (s)	space (Mb)
10	1	1
20	2	2
30	4	4
40	8	8
50	16	16
60	32	32
70	64	64
80	128	128
90	256	256
100	512	512

$2^{n/10}$

$2^{n/10}$

Algorithm 2

n	time (s)	space (Mb)
10	110	11
20	120	12
30	130	13
40	140	14
50	150	15
60	160	16
70	170	17
80	180	18
90	190	19
100	200	20

$100 + n$

$10 + n/10$

Algorithm 3

n	time (s)	space (Mb)
10	23	10000
20	30	10000
30	34	10000
40	37	10000
50	39	10000
60	41	10000
70	42	10000
80	44	10000
90	45	10000
100	46	10000

$10 \log n$

10000

Complexity of Algorithm

n	10	50	100	300	1000
$5n$	50	250	100	1500	5000
n^2	100	2500	10.000	90.000	10^7
n^3	1000	125.000	10^7	10^8	10^{10}
2^n	1024	10^{16}	10^{31}	10^{91}	10^{302}
$n!$	10^7	10^{65}	10^{161}	10^{623}	∞
n^n	10^{11}	10^{85}	10^{201}	10^{744}	∞

Atoms in the universe: 10^{78} – 10^{82}

Time since big-bang: 13.73 billion years = 10^{18} seconds

Complexity of Algorithm

Asymptotic growth of functions
Big-O notation

$$f \text{ is } O(g) \Leftrightarrow \exists k, c : f(n) \leq kg(n) \text{ for } n > c$$

Ordering of complexity

$O(1)$	$O(\log n)$	$O(\log^2 n)$	$O(\log^3 n)$	$O(\sqrt{n})$
$O(n)$	$O(n \log n)$	$O(n \log^2 n)$	$O(n \log^3 n)$	$O(n\sqrt{n})$
$O(n^2)$	$O(n^2 \log n)$	$O(n^2 \log^2 n)$	$O(n^2 \log^3 n)$	$O(n^2 \sqrt{n})$
$O(n^3)$	$O(n^4)$	$O(2^n)$	$O(n!)$	$O(n^n)$

Notice

$$O(100n^3) = O(n^3)$$

$$O(2n^4 + 100n^2 + n \log n) = O(n^4)$$

Exercises

Exercise 1 Write in big-O notation the asymptotic growth of the following functions

a $4n^3 + n^4 + n^3(\log n)^2$

b $100n + n \log n + \sqrt{n}$

c $5n^7 + 100n^6 + n^8$

d $n\sqrt{n} + n \log n + n \log(n^2)$